

DP Computer Science: B 4.1.1 ADTs in Programming (HL Only)

1. Core Concepts & Learning Objectives

Lesson 1: Introduction to ADTs

Learning Intentions

By the end of this lesson, students will be able to:

- Define what an **Abstract Data Type (ADT)** is
 - Explain the core principle of **abstraction** in the context of ADTs
 - Describe the purpose of ADTs in providing a **high-level description** of data
 - Identify the two key components of an ADT: **data and operations**
 - Understand why ADTs are used in programming
-

Key Vocabulary

Term	Definition
Abstract Data Type (ADT)	A high-level description of a data structure and its operations, independent of any specific implementation
Abstraction	The process of hiding implementation details and exposing only the essential features
Data	The information stored in the ADT
Operations	The actions or functions that can be performed on the data
Encapsulation	Bundling data and operations together, hiding internal state

Term	Definition
Implementation	The concrete code that realises the ADT using a specific data structure
Contract	The set of operations an ADT promises to provide, independent of how they are implemented

ATL Elements

Skill	Description
Thinking	Understanding the abstract nature of ADTs; making connections between concepts and their purpose
Communication	Discussing and explaining the definitions and purposes of ADTs
Self-Management	Organising notes and completing the worksheet within the allocated time

2. Introduction: What is an ADT?

What is Data?

Data is the fundamental information stored and processed by a computer program. It can be organised in many different ways.

The Need for Abstraction

text

THE NEED FOR ABSTRACTION

COMPLEXITY OF DATA STRUCTURES

- Arrays, linked lists, trees, graphs, stacks
- Each has its own implementation details
- Users don't need to know all details

SOLUTION: ABSTRACTION

- Hide implementation details
- Provide a high-level view
- Focus on "what" it does, not "how" it does it

What is an Abstract Data Type?

An Abstract Data Type (ADT) is a high-level description of a data structure and its operations, independent of any specific implementation. It separates the what (behaviour) from the how (implementation).

Two Key Components of an ADT

text

TWO KEY COMPONENTS OF AN ADT

DATA

- The information being stored
- The structure of the data
- Example: A list of items

OPERATIONS

- Actions that can be performed on the data
- Functions that manipulate the data
- Example: `add(item)`, `remove(item)`, `get(index)`

The ADT "Contract"

The operations of an ADT specify its behaviour and act as a contract for anyone using it. As long as the operations work as expected, users don't need to know how they are implemented.

text

THE ADT "CONTRACT"

ADT CONTRACT

"I promise to provide these operations..."

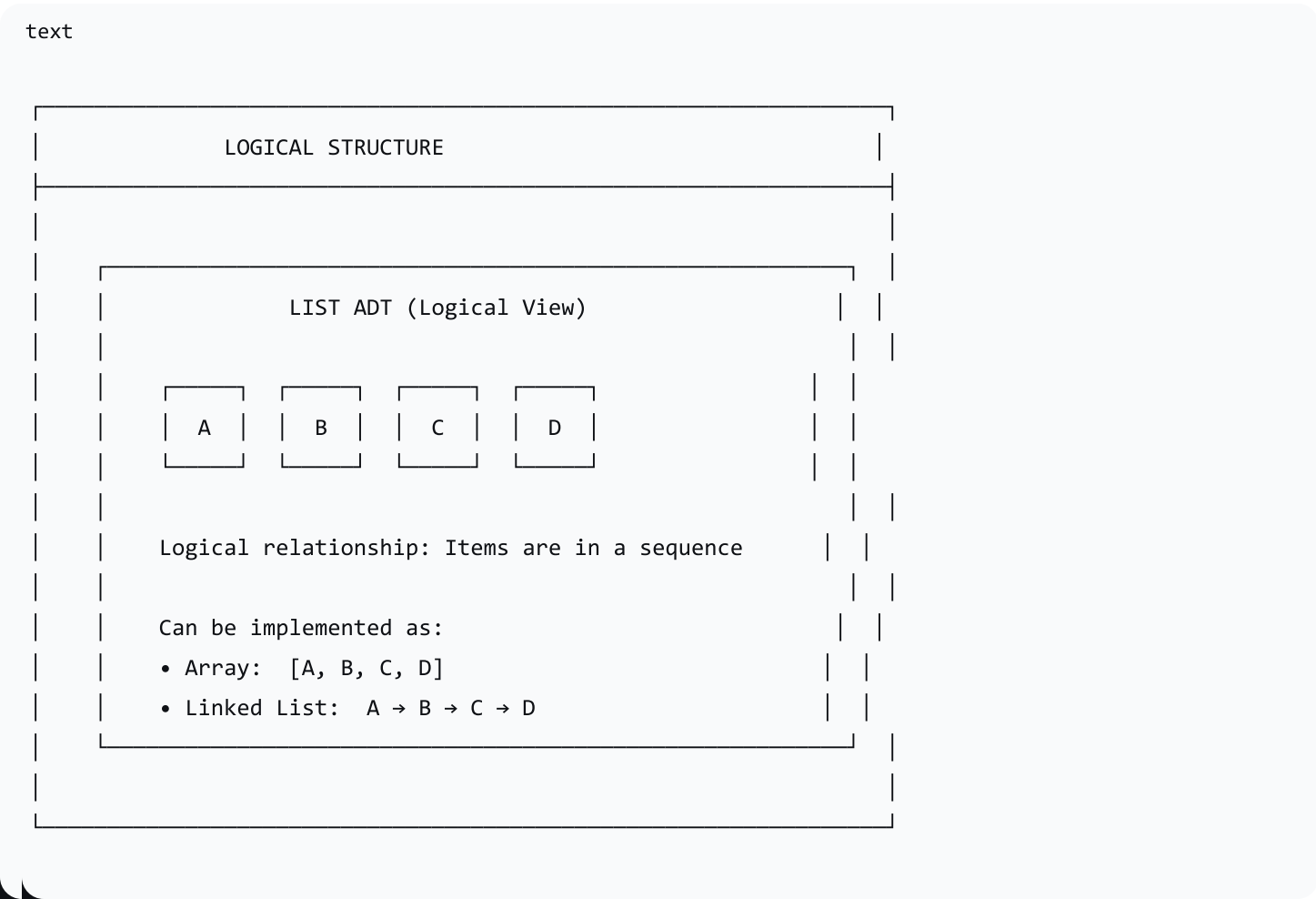
- `add(item)` → Adds an item to the collection
- `remove(item)` → Removes an item
- `contains(item)` → Checks if item exists
- `size()` → Returns the number of items

Users rely on these promises, NOT on how they are implemented!

3. Properties of ADTs

Logical Structure

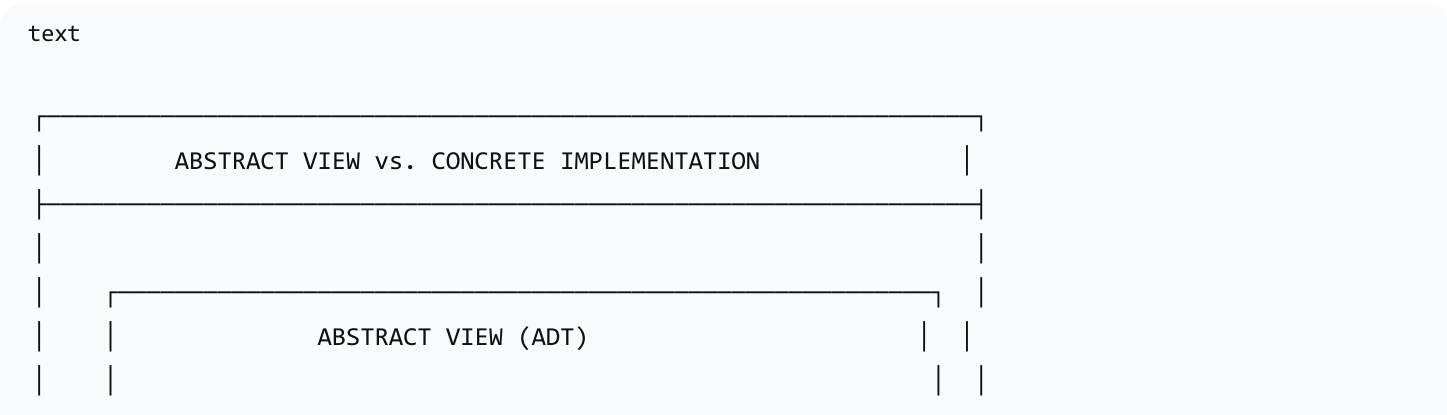
ADTs define the logical relationship between data elements, regardless of how they are physically stored.



Behaviour (Operations as a Contract)

The defined operations specify the ADT's behaviour and act as a contract for anyone using it.

Abstract View vs. Concrete Implementation



- What it does
- List: add, remove, get, size
- High-level description



CONCRETE IMPLEMENTATION

- How it does it
- Array implementation or Linked List
- Code-level details

The same ADT (List) can have multiple implementations!

4. Lesson 2: ADT Properties and Purpose

Learning Objectives

By the end of this lesson, students will be able to:

- Explain the properties of ADTs: **logical structure** and **behaviour**
- Distinguish between the abstract view of an ADT and its concrete implementation
- Understand the benefits of separating the abstract view from the implementation
- Appreciate how ADTs contribute to better program design

Benefits of Separation

text

BENEFITS OF SEPARATION

MAINTAINABILITY

- Code is easier to modify
- Implementation can be changed without affecting the rest of the program

FLEXIBILITY

- Can switch implementations for performance
- Can use different structures for different needs

ALGORITHM DESIGN

- Focus on solving problems at a high level
- Don't get bogged down in implementation details

REUSABILITY

- Same ADT can be used in different contexts
- Standardised operations across projects

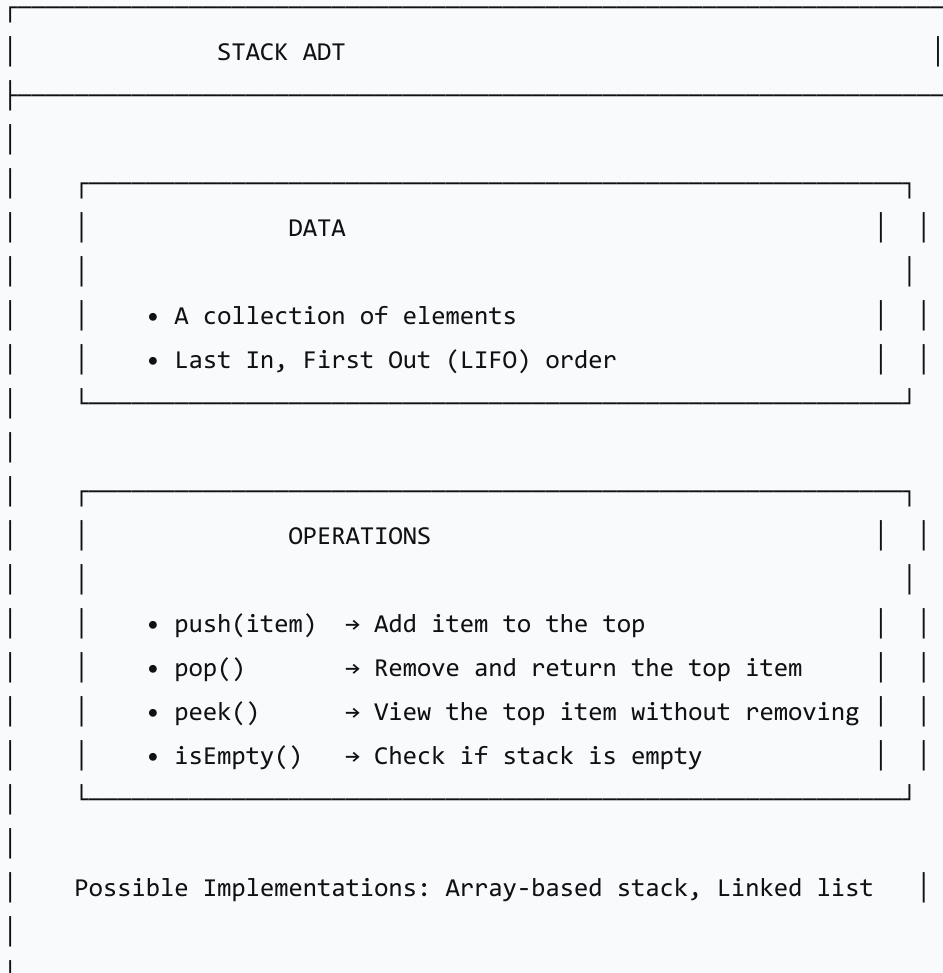
ADTs and Program Design

Thinking in terms of ADTs helps in designing well-structured and efficient programs by focusing on the essential data and operations needed to solve a problem.

5. ADT Examples

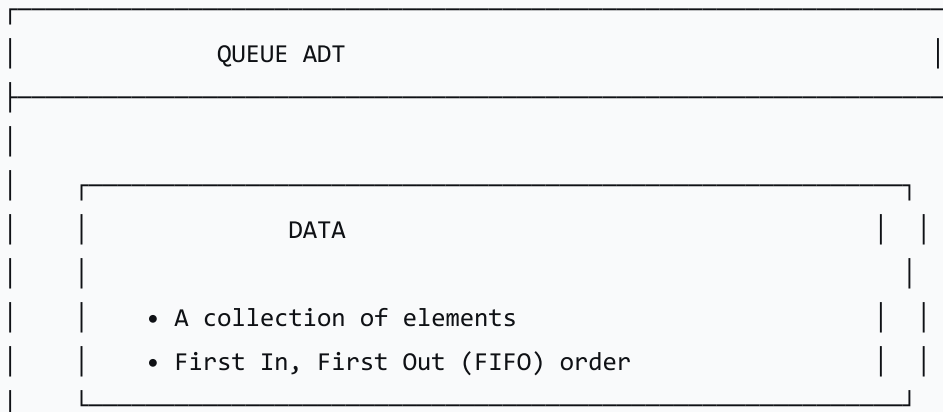
Example: Stack ADT

text



Example: Queue ADT

text



OPERATIONS

- enqueue(item) → Add item to the back
- dequeue() → Remove and return the front item
- front() → View the front item without removing
- isEmpty() → Check if queue is empty

Possible Implementations: Array-based queue, Linked list

Example: List ADT

text

LIST ADT

DATA

- An ordered collection of elements
- Elements can be accessed by position

OPERATIONS

- add(index, item) → Insert item at position
- get(index) → Get item at position
- remove(index) → Remove item at position
- size() → Get number of elements
- contains(item) → Check if item exists

Possible Implementations: Array-based list, Linked list



6. Worksheet Activities

Worksheet 1: Introduction to ADTs

Questions:

1. What does ADT stand for?
2. What are the two key components of an ADT?
3. Explain the concept of abstraction in the context of ADTs.
4. Why is it useful to separate the abstract view of an ADT from its concrete implementation?
5. Give an example of an ADT and describe its key operations.

Worksheet 2: ADT Properties and Purpose

Questions:

1. Describe the logical structure of a Stack ADT.
2. What is meant by the "behaviour" of an ADT?
3. Explain the difference between the abstract view and the concrete implementation of an ADT.
4. List three benefits of separating the abstract view from the implementation.
5. How do ADTs contribute to better program design?

7. Lesson Sequence

Lesson 1 Plan (45 minutes)

Phase	Time	Activity
Introduction	5 min	What are we learning today?

Phase	Time	Activity
What is Data?	10 min	Different ways data can be organised
The Need for Abstraction	10 min	Why we need a high-level view
Introduction to ADTs	15 min	Definition; data and operations; ADT contract
Wrap-up	5 min	Summary; worksheet introduction

Lesson 2 Plan (45 minutes)

Phase	Time	Activity
Recap	5 min	Review of ADT definition and components
Properties of ADTs	10 min	Logical structure, behaviour (contract)
Abstract vs. Concrete	15 min	Abstract view vs. implementation
Benefits & Program Design	10 min	Maintainability, flexibility, algorithm design
Wrap-up	5 min	Summary; worksheet introduction

8. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Worksheets	Evaluate understanding of ADT concepts
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Questions:

1. What is an Abstract Data Type?
 2. What are the two key components of an ADT?
 3. What is the purpose of an ADT contract?
 4. Why is abstraction important in programming?
-

9. Differentiation

Level	Strategy
Support	Use analogies; provide clear examples
Challenge	Explore more complex ADTs; compare implementations

10. Resources

- **Presentation** (Slide Deck)
 - **eBook** (B4.1.1 with examples)
 - **Worksheets:** ADT concepts and properties
 - **Worksheet Answers**
-

11. Summary: Key Takeaways

Concept	Key Point
ADT	High-level description of data and operations
Abstraction	Separating "what" from "how"
Data	The information stored in the ADT
Operations	Actions that can be performed on the data

Concept	Key Point
Contract	Promise of operations independent of implementation
Implementation	Concrete code realising the ADT

text

KEY REMINDERS

- ✓ ADT = Abstract Data Type
- ✓ Two components: Data + Operations
- ✓ Abstraction = hide implementation details
- ✓ Operations form a "contract"
- ✓ Same ADT can have multiple implementations
- ✓ ADTs improve maintainability and flexibility

"Abstract Data Types allow us to think about data at a higher level—focusing on what we can do with it, not how it is done."